

Development Plan

Production plan for the Tamil Nadu Government Agriculture Schemes RAG application.

Document purpose

This production-grade planning document defines scope, implementation controls, quality gates, security guardrails, and the operating workflow for the application. It reflects the current hardened build, including streaming answers, administrative access controls, and input/output guardrails.

Attribute	Value
Application	Tamil Nadu Agriculture Schemes Assistant
Source page	https://www.tn.gov.in/scheme_list.php?dep_id=Mg==
Current local schemes	54
Current indexed chunks	317
Chat model	gpt-4o-mini
Embedding model	text-embedding-3-small
Dataset hash prefix	603e7e7ae170
Answer delivery	Streaming, token-by-token (st.write_stream)
Admin controls	Env-gated: ADMIN_ACTIONS_ENABLED + ADMIN_PASSWORD

Prepared for implementation review, production readiness, and handoff.

1. Executive Summary

The application is a Streamlit-based retrieval assistant that helps users ask natural-language questions about publicly available Tamil Nadu Agriculture - Farmers Welfare Department schemes. It combines a defensive scraper, structured local storage, FAISS vector retrieval, OpenAI models, and optional LangSmith observability. Answers are now **streamed token-by-token** for responsiveness, generated under strict grounding, and bounded by input/output guardrails. Data-changing actions are protected by an environment-gated administrator mechanism.

Primary production objective

Deliver grounded, source-backed scheme answers while preventing homepage/navigation text, stale indexes, missing keys, prompt injection, formula-injected exports, or unverified model knowledge from polluting responses or harming users.

2. Product Scope

Area	In scope	Out of scope
Users	Farmers, student reviewers, administrators, demo evaluators	Official government service obligations
Data	Public scheme pages from approved tn.gov.in domains	Private data, protected endpoints, personal-data scraping

Area	In scope	Out of scope
Answers	Streamed, grounded answers from retrieved chunks with source URLs	Legal, financial, or eligibility guarantees beyond source data
Operations	Manual refresh, rebuild, troubleshooting, local artifacts	CI/CD, container, and managed cloud deployment (deferred)

3. Target Architecture

Streamlit UI	- >	Scraper	- >	JSON / CSV	- >	Documents	- >	FAISS	- >	Retriever	- >	Chat (stream)
--------------	-----	---------	-----	------------	-----	-----------	-----	-------	-----	-----------	-----	---------------

- Configuration is centralized in config.py; all secrets stay in environment variables. New typed settings cover admin gating, app environment, input/output caps, and request timeouts.
- scraper.py owns website access, normalization, deduplication, safe atomic persistence, and CSV formula-injection sanitization.
- rag_pipeline.py owns document formatting, chunking, dataset hashing, FAISS lifecycle, a shared retrieval helper, and both blocking and **streaming** answer generation under a strict, injection-resistant prompt.
- app.py owns the Streamlit interface, env-gated admin controls, streamed answer rendering inside the Assistant tab, session state, source display, and data browsing.

4. Delivery Roadmap

Phase	Goal	Key deliverables	Status
P0 Foundation	Runnable local app	Config, requirements, README, Streamlit shell	Done
P1 Data ingestion	Reliable public scrape	Session handling, retries, redirect detection, JSON/CSV	Done
P2 Retrieval	Reusable vector index	Documents, chunking, FAISS, metadata validation	Done
P3 Answering	Grounded assistant	Strict prompt, sources, unavailable-information behavior	Done
P4 UX hardening	Production interface	Readable controls, responsive layout, chat state, tabs	Done
P5 Observability	Trace critical runs	LangSmith project metadata and tags	Done
P6 Security & guardrails	Hardened build	Admin gating, CSV sanitization, input/token caps, prod error masking	Done
P7 Streaming UX	Responsive answers	Token-by-token streaming via st.write_stream + stream tests	Done
P8 Ops/Deployment	Deploy automation	CI, Docker, centralized logging, health check	Deferred

5. Workstreams And Controls

Workstream	Responsibilities	Quality controls
Frontend	Page structure, env-gated admin actions, streamed answer rendering, source expanders, CSV download	Desktop/mobile checks; contrast and layout review; admin controls hidden when disabled

Workstream	Responsibilities	Quality controls
Scraper	Approved-domain crawling, retries, cleaning, dedup, atomic writes, preserve last good dataset	Homepage detection; no overwrite with empty data; CSV formula-injection neutralized
RAG	Document conversion, chunking, hashing, FAISS persistence, retrieval, blocking + streaming answers	Index metadata validation; strict answer contract; safe streaming error fallback
Security	Secret handling, safe local FAISS load, admin gating, input bounding, prod error masking	No keys in UI/logs; admin password required in production; explicit deserialization boundary
Observability	LangSmith tracing and safe metadata for blocking and stream runs	Tracing can be disabled; no cookies/headers/secrets in metadata
Documentation	README, workflow, plan, env vars, admin controls, testing	Commands verified; model name and new variables documented

6. Engineering Backlog

Priority	Item	Status / rationale
Done	Admin guard / password for data-changing actions	Delivered: ADMIN_ACTIONS_ENABLED + ADMIN_PASSWORD gate refresh/rebuild
Done	Neutralize CSV formula injection on export	Delivered: leading = + - @ values prefixed with apostrophe
Done	Streaming answer output	Delivered: st.write_stream + answer_question_stream with stream tests
Done	Output-token, timeout, and input-length caps	Delivered: configurable caps reduce cost and injection surface
High	CI pipeline + container + health check	Deferred ops layer; required for managed deployment
High	Centralized secret-redacting logging	Replace basicConfig with rotating, redacting logging
Medium	Offline demo dataset mode	Keeps demos stable when the website blocks or redirects
Low	Multilingual answer mode	Improves accessibility for Tamil-speaking users

7. Testing Strategy

Layer	Checks	Pass signal
Static	py_compile, import checks, secret-placeholder scan, registered pytest markers	No syntax/import errors; markers resolve
Config	Valid config, missing key, invalid URL, bad chunk/timeout, production admin-password rule	Errors surfaced without leaking secrets
Scraper	Redirect, homepage marker, approved host, empty data, dedupe, previous-data preservation, CSV sanitization	Only valid records persisted; injection neutralized
Index	Dataset hash, model name, chunk size/overlap, index file existence, trusted-only load	Rebuild only when required
RAG	Grounded answer, unrelated question, source dedupe, prompt injection, empty-context no-call	Sources included; no unsupported claims

Layer	Checks	Pass signal
Streaming	Token generator yields, sources ready up front, empty retrieval -> no stream, mid-stream error fallback	Answer streams; safe degradation on error
UI	Rendering, action buttons, chat input, admin gating, streamed answer in Assistant tab	No overlap/contrast/state defects; no secret exposure

8. Release Gates

- OPENAI_API_KEY is configured outside source control; OPENAI_CHAT_MODEL verified against the current model list.
- LANGSMITH_TRACING is either validly configured or disabled.
- Scrape refresh produces non-empty JSON and CSV; CSV formula injection is neutralized.
- FAISS metadata matches dataset hash, embedding model, and chunk settings.
- At least five representative questions return grounded, streamed answers with sources.
- One unrelated question returns the approved unavailable-information response with no model call.
- Admin actions are disabled or password-protected; in production ADMIN_PASSWORD is set.
- Prompt-injection, XSS, and secret-exposure attempts remain safe.
- README, workflow PDF, and development plan PDF are delivered and consistent with the code.

9. Risk Register

Risk	Impact	Mitigation	Owner
Homepage redirect	Index unrelated content	Detect final URL/markers; preserve previous dataset	Scraper
HTML layout changes	Extraction quality drops	Selector fallbacks and fixture tests	Scraper
Missing/invalid OpenAI key	Chat and indexing fail	Friendly validation and setup docs	Config/UI
FAISS pickle loading	Unsafe deserialization	Load only app-generated local index; documented boundary	RAG
Model hallucination	Incorrect claims	Strict injection-resistant prompt; no outside knowledge; sources	RAG
Prompt injection	Secrets/instructions leak	Treat pages as data; refuse secrets; bounded input	RAG/Security
CSV formula injection	Malicious spreadsheet exec	Neutralize leading = + - @ in CSV export	Scraper/Security
Unprotected admin actions	Unplanned scrape/cost	Env-gated admin + production password requirement	Security
Unbounded cost/latency	Runaway tokens	Output-token, timeout, input-length, history caps	RAG/UI
Stale local data	Outdated answers	Expose scrape timestamp and refresh control	Operations

10. Operational Readiness

Runbook summary

Start with streamlit run app.py (or python app.py). Set APP_ENV, ADMIN_ACTIONS_ENABLED, and ADMIN_PASSWORD as needed. Refresh data only when needed, rebuild FAISS after every successful refresh, verify a sample streamed answer, and keep generated data and indexes local until the deferred ops/deployment layer is added.

Operation	Command or action	Expected result
Install	pip install -r requirements.txt (Python 3.11/3.12)	Runtime + dev/test deps installed from prebuilt wheels
Launch	streamlit run app.py	App opens; missing-key messages are friendly
Refresh	Sidebar -> Refresh Website Data (admin only)	New JSON/CSV saved only after a valid, non-empty scrape
Rebuild	Sidebar -> Rebuild FAISS Index (admin only)	index.faiss, index.pkl, and metadata updated
Ask	Type a question or pick an example	Answer streams token-by-token with a sources expander
Test	python -m pytest test/test_api.py -v	Backend + streaming tests pass with mocks only
Stop	Ctrl + C in terminal	Local server shuts down